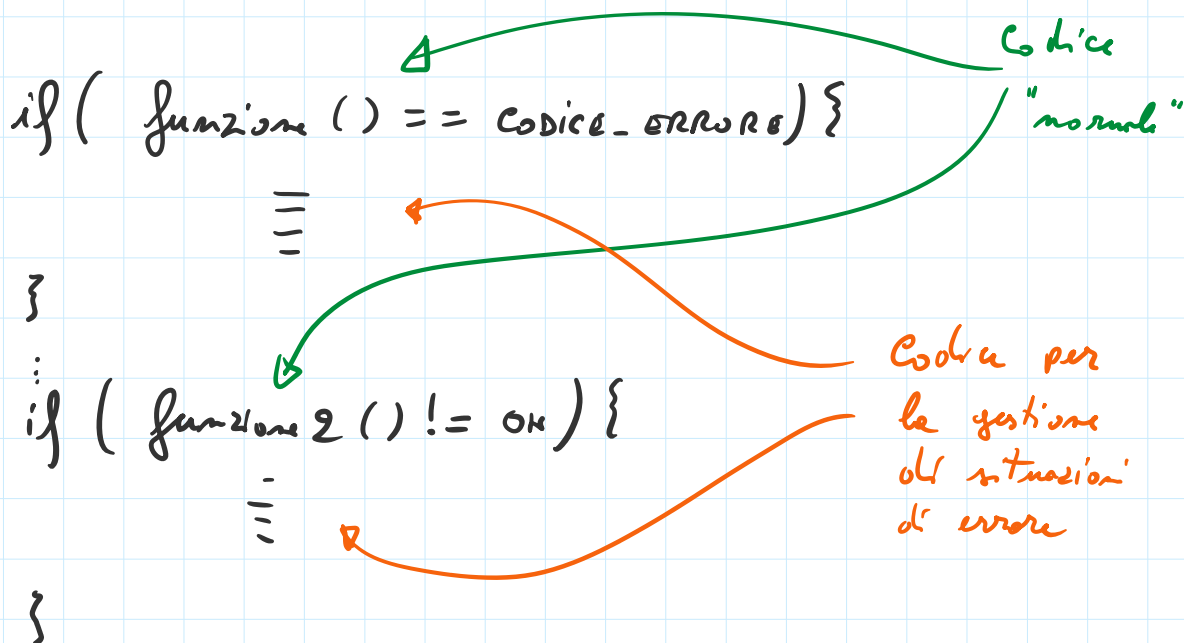


Eccezioni

Meccanismo utile alla rappresentazione di situazioni di errore e alla loro gestione

Senza eccezioni:



Il codice normale è "mischiato" a quello per la gestione degli errori.

Con le eccezioni è possibile mantenere separato il codice "normale" da quello per la gestione degli errori.

Alcune possibili situazioni di errore:

- 1) l'applicazione prova a scrivere in una porzione del filesystem su cui non ha i diritti di scrittura
- 2) l'applicazione comunica con il mondo esterno

- 2) l'applicazione comunica con il mondo esterno attraverso la rete e c'è una disconnessione
- 3) accedo a un array usando un indice non valido
- 4) chiamo un metodo su un riferimento nullo
- 5) c'è un errore nella JVM (il linking non ha successo)

Nelle situazioni 1) e 2) la JVM prevede azioni di recupero

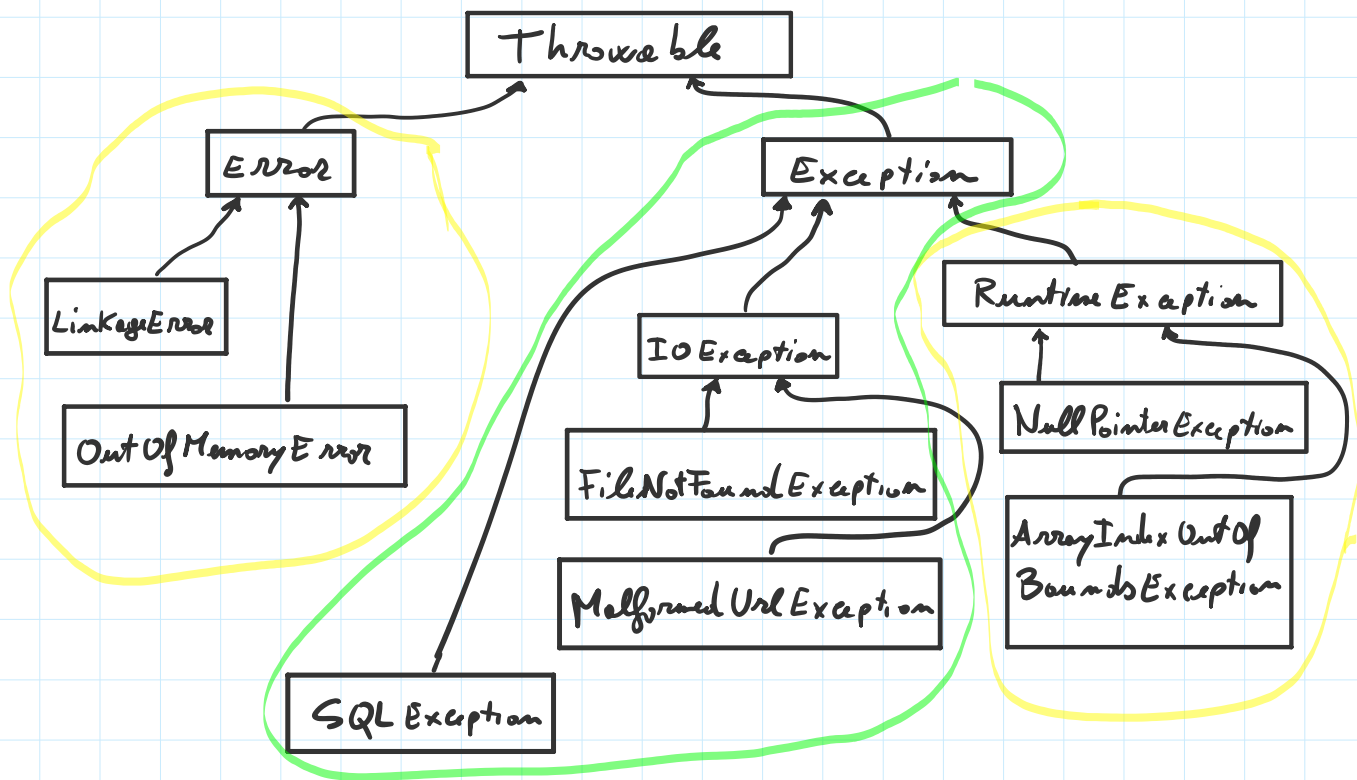
- chiedere all'utente se vuole salvare i dati in un altro punto del file system
- riprovare a connettersi dopo un po' di tempo

Nelle situazioni 3), 4) e 5) non ha senso prevedere azioni di recupero

- 3) e 4) sono errori logici, dobbiamo modificare il codice (il software)
- 5) è al di là della sfera di competenza del programmatore

In Java le situazioni di errore sono rappresentate come oggetti eccezione (istanze di opportune classi).

Le eccezioni sono organizzate in una gerarchia:



Eccezioni si dividono in
checked —
unchecked —

Il compilatore obbliga il programmatore a fornire codice di gestione per le eccezioni checked

Non c'è obbligo per le eccezioni unchecked

Due fasi fondamentali nella vita di un'eccezione

- il lancio
- la cattura

lanciamo un'eccezione quando rileviamo una situazione anomala (la situazione di errore)

Le eccezioni vengono catturate da gestori in cui specifichiamo cosa vogliamo fare (le azioni di recupero).

Come si lancia un'eccezione?
con l'istruzione throw

```
⋮  
if (situazione-di-errore) {  
    Exception e = new Exception();  
    throw e;  
}  
⋮
```

questo riferimento
e non mi serve
e niente quindi

in genere però le cose si scrivono così

```
⋮  
if (situazione-di-errore)  
    throw new Exception();  
⋮
```

Ho usato Exception per rappresentare la situazione di errore

- non ho usato Error perché rappresenta errori della JVM
- non ho usato Throwable perché troppo generico
- non ho usato RuntimeException perché rappresenta errori logici

Per catturare e gestire un'eccezione si usa il costrutto try-catch

```
try {  
    ===  
} catch (TipoEcc1 e) {  
    ===  
} catch (TipoEcc2 e2) {  
    ===  
}
```

} istruzioni che possono lanciare eccezioni

} il codice di gestione per la situazione di errore rappresentata da TipoEcc1

} codice di gestione per TipoEcc2

Se un'istruzione nel blocco try lancia un'eccezione l'esecuzione salta al blocco catch corrispondente poi vengono eseguite le istruzioni dopo l'ultimo blocco catch

```
try {  
    ===  
} catch (IOException ioe) {  
    ===  
} catch (SQLException sqle) {  
    ===  
}
```

istruzione che lancia IOException

} non vengono eseguite

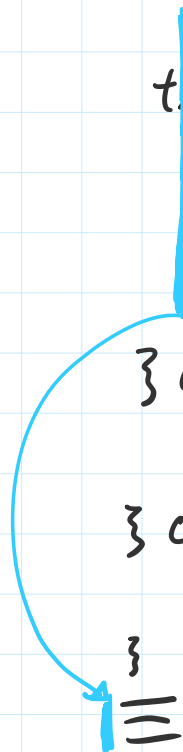
referimenti che posso usare per accedere all'oggetto eccezione

Tipi delle eccezioni che voglio catturare

in caso di esecuzione "normale"

in caso di eccezione "normale"

```
try {  
    // ...  
} catch (IOException ioe) {  
    // ...  
} catch (SQLException sqle) {  
    // ...  
}
```

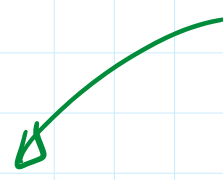


Quando viene lanciata un'eccezione viene eseguito il blocco catch corrispondente.

Corrispondente: il tipo dell'oggetto eccezione è un sottotipo di quanto indicato nel blocco catch

Quindi

```
try {  
    // ...  
} catch (Exception e) {  
    // ...  
}
```



questo blocco catch cattura tutti gli oggetti di Tipo Exception e sue sottoclassi (e quindi anche IOException e SQLException ..)

IOException e
SQL Exception ..)

Quando viene lanciata un'eccezione i blocchi catch vengono elaborati in sequenza

Non appena ne viene trovato uno corrispondente viene eseguito (gli altri, quelli a seguire, non vengono elaborati).

Questo comporta che non è possibile scrivere i blocchi catch in ordine qualunque quando i tipi di eccezioni catturate sono in una relazione supertipo-sottotipo

- Dobbiamo mettere prima i blocchi catch relativi ai tipi più specifici

```
try {  
    ==  
    ==
```

NO

```
} catch (Exception e){  
    ==
```

```
} catch (IOException ioe){  
    ==
```

```
}
```

non verrebbe
mai eseguito
perché il primo
"acchiappa" anche
IOException

OK

```
try {  
    ==  
    ==
```

, dal più

```

≡
} catch (IOException ioe) {
≡
} catch (Exception e) {
≡
}

```

dal più
specifico
al meno
specifico

Nel blocco catch posso accedere
all'oggetto eccezione

```
throw new Exception("Operazione non valida");
```

nel blocco catch:

```

} catch (Exception e) {
    String s = e.getMessage();
    System.err.println(s);
}

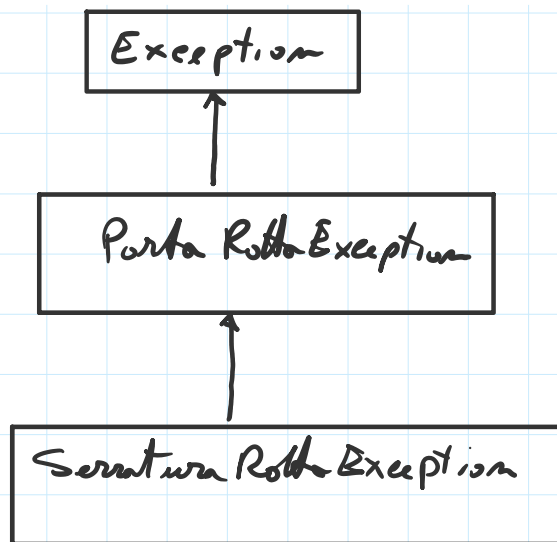
```

recupero
il messaggio

Il programmatore può definire nuovi
tipi di eccezioni

Tipicamente viene estesa la classe `Exception`

Esempio:



```
public class PortaRottaException extends Exception {  
    public PortaRottaException() {  
        super();  
    }  
    public PortaRottaException(String m) {  
        super(m);  
    }  
}
```

```
public class SerenaturaRottaException extends  
    PortaRottaException {  
    public SerenaturaRottaException() {  
        super();  
    }  
    public SerenaturaRottaException(String s) {  
        super(s);  
    }  
}
```

Per lanciare :

```
if (situazione di errore)
    throw new PortoRottoException();
```

In effetti il blocco try-catch
può prevedere anche un blocco finally

```
try {
    ==
    ==
    ==
} catch (Tipo1 t1) {
    ==
} catch (Tipo2 t2) {
    ==
} finally {
    ==
}
```

} queste istruzioni vengono
eseguite sempre
(sia in caso di eccezioni
sia se tutto normale)

Se tutto "normale"

```
try {
    ==
    ==
    ==
} catch ( — ) {
    ==
} catch ( — ) {
    ==
} finally {
    ==
}
```

```

} finally {
}

```

in presenza di eccezioni anomale

```

try {
} catch ( — ) {
} catch ( — ) {
} finally {
}

```

In pratica il codice del blocco finally viene eseguito sempre
- spesso contiene del codice di pulizia

```

import java.io.*;

class Main {
    public static void main(String[] args) {
        Writer w = null;
        try {
            w = new FileWriter("out.txt");
            for(int i=0; i<10; i++)
                w.write(String.valueOf(i) + System.getProperty("line.separator"));
        } catch (IOException ioe) {
            System.out.println("Errore: " + ioe.getMessage());
        } finally {
            try {
                if(w != null)
                    w.close();
            } catch (IOException e){
                // EAT
            }
        }
    }
}

```

rimuovi

```

    }
  }
} // EAT

```

questo è uno dei pochi casi in cui siamo autorizzati a scrivere un blocco catch vuoto

[PA2021 ecc2](#)

The screenshot shows an IDE with a file named `Main.java` and a console window. The code in `Main.java` is as follows:

```

1 import java.io.*;
2
3 class Main {
4     public static void main(String[] args) {
5         Writer w = null;
6         try {
7             w = new FileWriter("out.txt");
8             for(int i=0; i<10; i++)
9                 w.write(String.valueOf(i) + System.getProperty(
10                     "line.separator"));
11         } catch (IOException ioe) {
12             System.out.println("Errore: " + ioe.getMessage());
13         } finally {
14             try {
15                 if(w != null)
16                     w.close();
17             } catch (IOException e){
18                 // EAT
19             }
20         }
21     }
22 }

```

The console output shows the command to run the program and the resulting file content:

```

> javac -classpath ./run_dir/junit-4.12.jar:./run_dir/hamcrest-1.3.jar:./run_dir/json-simple-1.1.1.jar -d . Main.java
> java -classpath ./run_dir/junit-4.12.jar:./run_dir/hamcrest-core-1.3.jar:./run_dir/json-simple-1.1.1.jar Main
> more out.txt
0
1
2
3
4
5
6
7
8
9

```

Attenzione: non lasciare mai vuoti i blocchi catch!



```

try {
    ≡ ← X
} catch ( ... ) {
    ≡
}

```

Eccezioni e metodi

Supponiamo che nel corpo di un metodo ci siano istruzioni che possono lanciare eccezioni

2 possibilità:

- gestiamo la situazione anomala localmente con un blocco try-catch
- propaghiamo l'eccezione al chiamante

Certe eccezioni possono "fuoriuscire" da un metodo (quelle che abbiamo deciso di non gestire localmente).

Le eccezioni che possono fuoriuscire devono essere indicate con la clausola throws

```

public class Pila {
    ≡
    public Object pop() throws PilaVuotaException {
        if (empty())
            throw new PilaVuotaException();
    }
}

```

```

    if (empty())
        throw new PilaVuotaException();
    = } queste istruzioni non
      } vengono eseguite se
    } viene lanciata PilaVuotaException

```

la "catena" di chiamate di metodo viene percorsa a ritroso fino a trovare un blocco catch in grado di acchiappare l'eccezione

Se un metodo può lanciare più eccezioni (di tipo diverso) basta specificarle tutte dopo il throws

```

void m() throws IOException, SQLException {
    =
}

```

Il ragionamento vale per le eccezioni che possono fuoruscire

```

void m1() throws IOException {

```

```

    try {
        // ...
    } catch (SQLException se) {
        // ...
    }

```

se tutte le istruzioni che possono generare `SQLException` sono qui allora non possono fuorviare

le eccezioni che possono lanciare `IOException` possono essere in un punto qualunque

Per le eccezioni checked la gestione è obbligatoria:

- o propagare
- o gestione locale

```

void salva() throws IOException {
    write();
}

```

se lancia eccezione di tipo `IOException` le istruzioni rimanenti non vengono eseguite.

Non c'è obbligo per quelle unchecked

```

void v() {
    // ...
}

```

```
int[] x = new int[1];  
x[0] = 1;  
o.m();  
}
```

Esempio

[PA2021 ecc1](#)